



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS
POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Practica 4

Configuración y Parpadeo de LED

con SDK de C/C++ para Raspberry Pi Pico W

Carrera:

Ingeniería Mecatrónica - Cuarto Semestre

Alumno:

Saúl Martínez Almazán

Docente:

Ing. Luis Jesús Padrón Padrón

Fecha:

24 de Febrero de 2026

Índice

1. Introducción	2
1.1. Objetivo	2
1.2. Importancia del SDK de Raspberry Pi	2
1.3. Aplicaciones del Pico W	2
2. Desarrollo	2
2.1. Configuración del Entorno de Desarrollo	2
2.2. Creación del Proyecto	3
2.3. Proceso de Compilación y Carga	3
3. Resultados	3
3.1. Código Fuente Implementado	3
3.2. Evidencias y Solución de Problemas	4
4. Conclusión	6
4.1. Reflexión	6
4.2. Mejoras Propuestas	6
4.3. Aplicación Futura	6

1. Introducción

En esta sección se presenta el contexto, objetivo e importancia del presente reporte, el cual documenta el proceso de configuración del entorno de desarrollo para el microcontrolador Raspberry Pi Pico W utilizando el SDK oficial de C/C++.

1.1. Objetivo

Establecer y validar el entorno de desarrollo para el Raspberry Pi Pico W utilizando el SDK oficial en C/C++, culminando con la implementación de un programa de control de hardware básico (Blink).

1.2. Importancia del SDK de Raspberry Pi

A diferencia de entornos interpretados como MicroPython, el SDK de C/C++ permite un acceso de bajo nivel al hardware del microcontrolador RP2040, optimizando el uso de memoria y la velocidad de ejecución, lo cual es crítico en sistemas de tiempo real.

1.3. Aplicaciones del Pico W

Gracias a su chip CYW43439, el Pico W es fundamental en el desarrollo de aplicaciones de IoT (Internet de las Cosas), permitiendo la creación de:

- Servidores web embebidos
- Nodos de sensores inalámbricos
- Sistemas de control remoto de alta eficiencia

2. Desarrollo

2.1. Configuración del Entorno de Desarrollo

La preparación del sistema incluyó la instalación de herramientas esenciales: ARM GCC Toolchain, CMake, Python 3 y Git.

Librerías Utilizadas:

- **pico_stdlib:** Librería estándar que agrupa funciones comunes como el control de GPIO y retardos (`sleep_ms`).
- **pico_cyw43_arch_none:** Librería específica para el modelo Pico W. Es necesaria porque, en este modelo, el LED integrado no está conectado directamente al procesador RP2040, sino al chip de Wi-Fi.

Estructura del Proyecto:

Se creó un directorio raíz que contiene la siguiente organización:

- `build/` — Carpeta para archivos temporales de compilación
- `main.c` — Código fuente principal
- `CMakeLists.txt` — Archivo de configuración del sistema de compilación

2.2. Creación del Proyecto

Terminal de Desarrollador:

Se utilizó el *Pico – Developer PowerShell*, el cual preconfigura las rutas del SDK (`PICO_SDK_PATH`) automáticamente.

Archivo `CMakeLists.txt`:

Este archivo es el corazón de la compilación. Debe:

1. Declarar la versión de CMake requerida
2. Incluir el archivo `pico_sdk_import.cmake`
3. Vincular las librerías `pico_stdlib` y `pico_cyw43_arch_none`

Código en C:

Se implementó una rutina que inicializa la arquitectura de comunicación con el chip de red para poder conmutar el estado del pin del LED.

2.3. Proceso de Compilación y Carga

1. **Compilación en VS Code:** Se ejecutó el comando `cmake ..` dentro de la carpeta `build` para generar los archivos de construcción, seguido de `nmake` o `make` para compilar el binario.
2. **Generación del `.uf2`:** El proceso de compilación finaliza creando un archivo con extensión `.uf2`, listo para ser cargado al microcontrolador.
3. **Transferencia:** Se conectó el microcontrolador manteniendo presionado el botón `BOOTSEL`, lo que monta al dispositivo como una unidad de almacenamiento masivo. El archivo `.uf2` se arrastró a dicha unidad.
4. **Verificación:** El programa se ejecuta instantáneamente tras la carga, confirmando el parpadeo cíclico del LED verde integrado.

3. Resultados

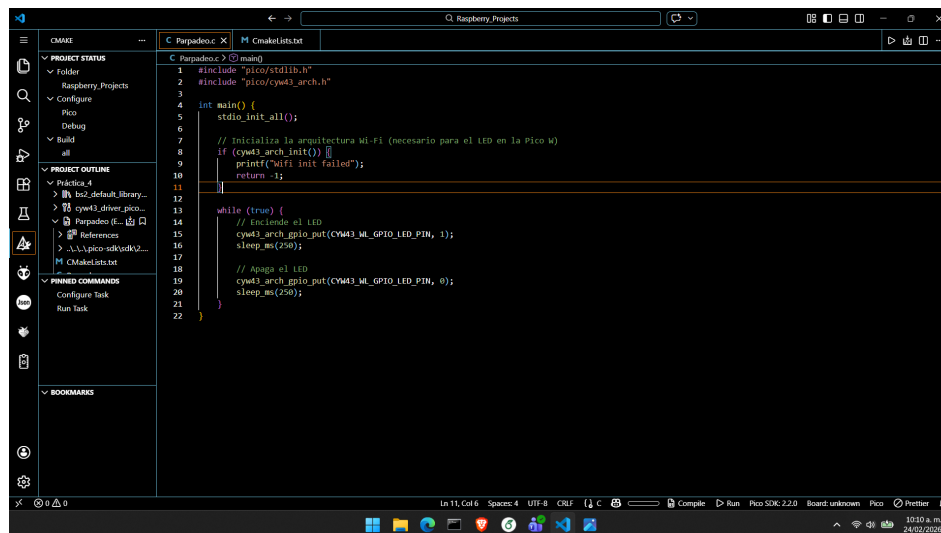
3.1. Código Fuente Implementado

A continuación se presenta el código fuente en C desarrollado para el control del LED integrado del Raspberry Pi Pico W:

```
1 #include "pico/stdlib.h"
2 #include "pico/cyw43_arch.h"
3
4 int main() {
5     stdio_init_all();
6     // Inicializacion necesaria para el hardware del Pico W
7     if (cyw43_arch_init()) {
8         return -1;
9     }
10    while (true) {
11        // Encender LED integrado
12        cyw43_arch_gpio_put(CYW43_WL_GPIO_LED_PIN, 1);
13        sleep_ms(500);
14        // Apagar LED integrado
15        cyw43_arch_gpio_put(CYW43_WL_GPIO_LED_PIN, 0);
16        sleep_ms(500);
17    }
18 }
```

3.2. Evidencias y Solución de Problemas

Capturas de Pantalla:



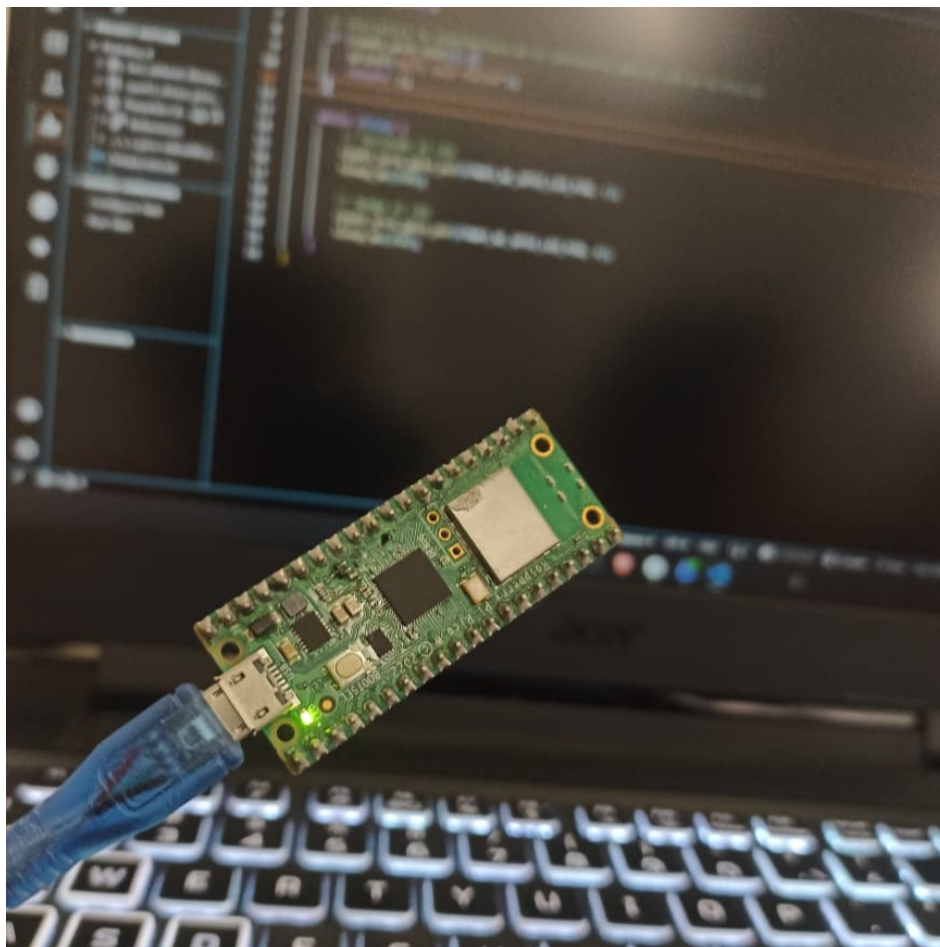


Figura 1: Unidad RPI-RP2 detectada tras conectar el Pico W en modo BOOTSEL

Errores Comunes y Soluciones:

Cuadro 1: Errores encontrados y soluciones aplicadas durante el desarrollo

Error Encontrado	Solución Aplicada
El código no compila por falta de <code>PICO_SDK_PATH</code> .	Se configuró la variable de entorno global en Windows.
El LED no enciende usando <code>gpio_put(25, 1)</code> .	Se corrigió el código para usar las funciones de <code>cyw43_arch</code> , ya que el LED del Pico W se controla mediante el bus del chip Wi-Fi.

4. Conclusión

4.1. Reflexión

El uso del SDK en C permite una comprensión mucho más profunda de la arquitectura del microcontrolador y el flujo de trabajo profesional basado en herramientas de compilación cruzada. Este enfoque de bajo nivel es indispensable para el desarrollo de sistemas embebidos robustos y eficientes.

4.2. Mejoras Propuestas

Se podría implementar el uso de *timers* de hardware o interrupciones para evitar el uso de `sleep_ms`, lo cual presenta las siguientes ventajas:

- **Multitarea:** El procesador puede realizar otras tareas mientras el LED espera su siguiente cambio de estado.
- **Precisión temporal:** Los timers de hardware ofrecen mayor exactitud en los intervalos de tiempo.
- **Eficiencia energética:** El procesador puede entrar en modos de bajo consumo entre eventos.

4.3. Aplicación Futura

Este flujo de trabajo es la base para proyectos complejos, como el sistema de visión “*The Sentinel*”, donde la eficiencia en el procesamiento de datos será crucial para el éxito del sistema de control.

Referencias

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.